

```

# -*- coding: utf-8 -*-
"""
قابلة Kivy (تطبيق مواعيد مباريات كرة القدم - نسخة
Buildozer) للتحويل إلى أندرويد عبر

المزايا:
- قاعدة بيانات دوريات/أندية/ملاعب قابلة للتوسع
(leagues_data.json)
- عند اختيار الفريق يُقترح ملعبه تلقائياً (قابل للتعديل)
- باسم logos/ عرض شعار كل نادٍ إن وُجد ملفه داخل مجلد
الفريق
وإلا يُعرض شعار بديل بأول (logos/الهلال.png: مثال)
حرف من اسم الفريق
أو (plyer) تنبيه قبل المباراة عبر إشعار نظام أندرويد
- نافذة داخل التطبيق كبدل
matches.json حفظ البيانات في -
"""

import os
import json
import threading
from datetime import datetime, timedelta

from kivy.app import App
from kivy.clock import Clock
from kivy.core.window import Window
from kivy.metrics import dp
from kivy.properties import StringProperty
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button

```

```

from kivy.uix.gridlayout import GridLayout
from kivy.uix.image import Image
from kivy.uix.label import Label
from kivy.uix.popup import Popup
from kivy.uix.scrollview import ScrollView
from kivy.uix.spinner import Spinner
from kivy.uix.textinput import TextInput
from kivy.graphics import Color, Ellipse

BASE_DIR =
os.path.dirname(os.path.abspath(__file__))
LEAGUES_FILE = os.path.join(BASE_DIR,
"leagues_data.json")
MATCHES_FILE = os.path.join(BASE_DIR,
"matches.json")
LOGOS_DIR = os.path.join(BASE_DIR, "logos")

REMINDER_MINUTES = 30
CHECK_INTERVAL_SECONDS = 30

try:
    from plyer import notification
    HAS_PLYER = True
except Exception:
    HAS_PLYER = False

# ----- طبقة البيانات -----
-----

def load_leagues():
    with open(LEAGUES_FILE, "r",
encoding="utf-8") as f:
        return json.load(f)

```

```

def load_matches():
    if os.path.exists(MATCHES_FILE):
        with open(MATCHES_FILE, "r",
encoding="utf-8") as f:
            return json.load(f)
    return []

def save_matches(matches):
    with open(MATCHES_FILE, "w",
encoding="utf-8") as f:
        json.dump(matches, f,
ensure_ascii=False, indent=4)

def find_team_stadium(leagues, league_name,
team_name):
    for team in leagues.get(league_name,
[]):
        if team["name"] == team_name:
            return team["stadium"]
    return ""

def logo_path_for(team_name):
    """ يبحث عن ملف شعار بامتدادات شائعة، وإلا يرجع ""
None. """
    for ext in (".png", ".jpg", ".jpeg"):
        path = os.path.join(LOGOS_DIR,
team_name + ext)
        if os.path.exists(path):
            return path

```

```

        return None

# ----- ودجت شعار الفريق (مع -----
# ----- بديل تلقائي) -----

class TeamLogo(BoxLayout):
    """
    يعرض صورة الشعار إن وُجدت، وإلا دائرة ملونة بأول
    حرف من اسم الفريق.
    """

    def __init__(self, team_name,
                 size_dp=44, **kwargs):
        super().__init__(size_hint=(None,
None), size=(dp(size_dp), dp(size_dp)),
**kwargs)
        path = logo_path_for(team_name)
        if path:

self.add_widget(Image(source=path))
        else:

self.add_widget(self._placeholder(team_name
, size_dp))

    def _placeholder(self, team_name,
size_dp):
        label = Label(text=(team_name[:1]
if team_name else "?"), bold=True,
color=(1, 1, 1, 1))
        with label.canvas.before:
            # لون ثابت مبني على اسم الفريق حتى يكون #
            كل فريق بلون مميز تقريباً
            seed = sum(ord(c) for c in
team_name) if team_name else 1

```

```

        r = (seed % 50) / 100 + 0.3
        g = (seed % 70) / 140 + 0.2
        b = (seed % 90) / 120 + 0.25
        Color(r, g, b, 1)
        self._ellipse =
Ellipse(pos=label.pos, size=(dp(size_dp),
dp(size_dp)))

label.bind(pos=self._update_ellipse,
size=self._update_ellipse)
        return label

    def _update_ellipse(self, instance,
value):
        self._ellipse.pos = instance.pos
        self._ellipse.size = instance.size

# ----- عنصر عرض مباراة واحدة -----
-----

class MatchRow(BoxLayout):
    def __init__(self, match, on_delete,
**kwargs):

super().__init__(orientation="horizontal",
size_hint_y=None, height=dp(64),
                    spacing=dp(8),
padding=dp(6), **kwargs)

self.add_widget(TeamLogo(match["team1"]))
        info =
BoxLayout(orientation="vertical")

```

```

        info.add_widget(Label(text=f"
{match['team1']}  {match['team2']}\"",
bold=True))

        info.add_widget(Label(text=f"
{match['date']} - {match['time']} |
{match['stadium']}\"",

font_size="12sp"))
        self.add_widget(info)

self.add_widget(TeamLogo(match["team2"]))

        del_btn = Button(text="حذف",
size_hint=(None, None), size=(dp(60),
dp(40)))
        del_btn.bind(on_release=lambda *_:
on_delete(match["id"]))
        self.add_widget(del_btn)

# ----- الشاشة الرئيسية -----
-----

class RootLayout(BoxLayout):
    def __init__(self, **kwargs):

super().__init__(orientation="vertical",
padding=dp(10), spacing=dp(8), **kwargs)

        self.leagues = load_leagues()
        self.matches = load_matches()

        self._build_form()
        self._build_list()

```

```

        self.refresh_list()

        # مراقبة التنبيهات في خيط خلفي
        self.stop_event = threading.Event()

        threading.Thread(target=self._reminder_loop
            , daemon=True).start()

        # ----- نموذج الإضافة -----
        def _build_form(self):
            form = GridLayout(cols=2,
                size_hint_y=None, height=dp(280),
                spacing=dp(6))

            form.add_widget(Label(text="الدوري"))
            self.league_spinner =
                Spinner(text="اختر الدوري",
                    values=list(self.leagues.keys()))

            self.league_spinner.bind(text=self._on_league_change)

            form.add_widget(self.league_spinner)

            form.add_widget(Label(text="الفريق الأول"))
            self.team1_spinner =
                Spinner(text="اختر الفريق", values=[])

            self.team1_spinner.bind(text=self._on_team_change)
            form.add_widget(self.team1_spinner)

            form.add_widget(Label(text="الفريق

```

```

        الثاني"))
        self.team2_spinner =
Spinner(text="اختر الفريق", values=[])

self.team2_spinner.bind(text=self._on_team_
change)
        form.add_widget(self.team2_spinner)

        form.add_widget(Label(text="الملعب
(تلقائي/قابل للتعديل)"))
        self.stadium_input =
TextInput(multiline=False)
        form.add_widget(self.stadium_input)

        form.add_widget(Label(text="التاريخ
(YYYY-MM-DD)"))
        self.date_input =
TextInput(multiline=False)
        form.add_widget(self.date_input)

        form.add_widget(Label(text="الوقت
(HH:MM)"))
        self.time_input =
TextInput(multiline=False)
        form.add_widget(self.time_input)

        add_btn = Button(text="✚ إضافة
المباراة", size_hint_y=None, height=dp(44))

add_btn.bind(on_release=self.add_match)

        self.add_widget(form)
        self.add_widget(add_btn)

```



```

def _on_league_change(self, spinner,
league_name):
    teams = [t["name"] for t in
self.leagues.get(league_name, [])]
    self.team1_spinner.values = teams
    self.team2_spinner.values = teams
    self.team1_spinner.text = "اختر
الفريق"
    self.team2_spinner.text = "اختر
الفريق"

def _on_team_change(self, *_):
    # يقترح ملعب الفريق الأول تلقائياً عند اختياره
    league = self.league_spinner.text
    team1 = self.team1_spinner.text
    if team1 and team1 != "اختر الفريق":
        stadium =
find_team_stadium(self.leagues, league,
team1)
        if stadium:
            self.stadium_input.text =
stadium

    # ----- قائمة المباريات -----
    def _build_list(self):
        self.list_box = GridLayout(cols=1,
size_hint_y=None, spacing=dp(4))

self.list_box.bind(minimum_height=self.list
_box.setter("height"))

    scroll = ScrollView()
    scroll.add_widget(self.list_box)
    self.add_widget(Label(text="17

```

```

المباريات القادمة", size_hint_y=None,
height=dp(30), bold=True))
        self.add_widget(scroll)

    def refresh_list(self):
        self.list_box.clear_widgets()
        sorted_matches = sorted(
            self.matches,
            key=lambda m:
datetime.strptime(f"{m['date']}
{m['time']}", "%Y-%m-%d %H:%M"),
        )
        for m in sorted_matches:

self.list_box.add_widget(MatchRow(m,
on_delete=self.delete_match))

# ----- الإضافة والحذف -----
    def add_match(self, *_):
        team1 = self.team1_spinner.text
        team2 = self.team2_spinner.text
        stadium =
self.stadium_input.text.strip()
        date_str =
self.date_input.text.strip()
        time_str =
self.time_input.text.strip()

        if "اختار" in team1 or "اختار" in team2
or not stadium or not date_str or not
time_str:
            self._show_message("الرجاء", "
تنبيه", "تعبئة جميع الحقول واختبار الفريقين")
            return

```

```

        try:
            datetime.strptime(f"{date_str}
{time_str}", "%Y-%m-%d %H:%M")
        except ValueError:
            self._show_message("خطأ", "صيغة
مثال: 01-09-2026 و التاريخ أو الوقت غير صحيحة
18:30")

        return

        new_id = (max([m["id"] for m in
self.matches], default=0) + 1)
        match = {
            "id": new_id,
            "team1": team1,
            "team2": team2,
            "date": date_str,
            "time": time_str,
            "stadium": stadium,
            "notified": False,
        }
        self.matches.append(match)
        save_matches(self.matches)
        self.refresh_list()
        self._show_message("تمت إضافة", f"مباراة: {team1} vs {team2}")

    def delete_match(self, match_id):
        self.matches = [m for m in
self.matches if m["id"] != match_id]
        save_matches(self.matches)
        self.refresh_list()

# ----- التنبيهات -----

```

```

def _reminder_loop(self):
    while not self.stop_event.is_set():
        self._check_due_reminders()

self.stop_event.wait(CHECK_INTERVAL_SECONDS
)

def _check_due_reminders(self):
    now = datetime.now()
    changed = False
    for m in self.matches:
        if m.get("notified"):
            continue
        match_dt = datetime.strptime(f"
{m['date']} {m['time']}", "%Y-%m-%d %H:%M")
        remind_at = match_dt -
timedelta(minutes=REMINDER_MINUTES)
        if remind_at <= now < match_dt:
            m["notified"] = True
            changed = True
            Clock.schedule_once(lambda
dt, mm=m: self._fire_reminder(mm))
        if changed:
            save_matches(self.matches)

def _fire_reminder(self, match):
    title = "🕒 تذكير بمباراة قادمة"
    message = (f"{match['team1']} vs
{match['team2']}\n"
               f"الساعة {match['time']}
في {match['stadium']}")
    if HAS_PLYER:
        try:

```

```
notification.notify(title=title,
message=message, timeout=10)
        return
    except Exception:
        pass
    self._show_message(title, message)

    def _show_message(self, title,
message):
        popup = Popup(title=title,

content=Label(text=message),
                    size_hint=(0.8,
0.4))
        popup.open()

class FootballApp(App):
    def build(self):
        Window.clearcolor = (0.97, 0.97,
0.97, 1)
        return RootLayout()

if __name__ == "__main__":
    FootballApp().run()
```